HTML & CSS • MODULE 07

Responsive Design

Build for Every Screen and Every User

■ RESPONSIVE DESIGN

■ LEARNING OUTCOME

By the end of this module you will build layouts that look and work perfectly on every screen size — from 320px phones to 4K monitors — using mobile-first media queries, fluid typography, and responsive images.

■ Skills You Will Gain

- Write mobile-first media queries with min-width breakpoints
- Use relative units: em, rem, %, vw, vh, vmin, vmax
- Create fluid typography with clamp()
- Implement responsive images with srcset and picture
- Define and use CSS custom properties (variables)
- Apply responsive design patterns: stack, sidebar, card grid

■ **Over 60% of global web traffic is from mobile devices. A non-responsive website is inaccessible to the majority of your users. Responsive design is not optional — it's the baseline.**

SECTION 1

Viewport and Mobile-First Approach

Why Mobile-First?

DESKTOP-FIRST: Write desktop styles, then undo/override for mobile. Problem: You end up fighting your own CSS to simplify for mobile. MOBILE-FIRST: Write simple mobile styles first, then ENHANCE for larger screens. Benefit: Mobile CSS is simpler. Progressive enhancement. Better performance on slow connections. Mobile-first uses min-width media queries: @media (min-width: 768px) { /* tablet and up */ } @media (min-width: 1024px) { /* desktop and up */ }

```
/* STEP 1: Meta viewport tag in HTML head — REQUIRED */ <meta name="viewport"
content="width=device-width, initial-scale=1.0"> /* STEP 2: Mobile-first CSS */ /* Base styles —
mobile (320px and up) */ .container { padding: 16px; width: 100%; } /* Tablet (768px and up) */
@media (min-width: 768px) { .container { padding: 24px; } } /* Desktop (1024px and up) */ @media
(min-width: 1024px) { .container { padding: 48px; max-width: 1200px; margin: 0 auto; } } /* Large
desktop (1440px and up) */ @media (min-width: 1440px) { .container { max-width: 1400px; } } /*
Common breakpoint reference: 320px: Small phones (iPhone SE) 375px: Standard phones (iPhone 14)
768px: Tablets (iPad portrait) 1024px: Large tablets / small laptops 1280px: Laptops 1440px:
Desktops 1920px: Full HD monitors */
```

SECTION 2

CSS Units — Relative vs Absolute

	Relative To	Best For
	Fixed pixels (absolute)	Borders, box shadows, fine details — NOT font size o
	Parent element's font-size	Component-level spacing that scales with its own font
	Root (html) element's font-size	Font sizes, consistent spacing throughout entire site
	Parent element's size	Fluid widths, responsive layouts
	1% of viewport width	Full-width elements, fluid typography
	1% of viewport height	Hero sections, full-screen overlays
	1% of smaller viewport dimension	Elements that must fit on screen
	1% of larger viewport dimension	Large typography that fills screen
	Width of '0' character	Input widths (e.g. 20ch for 20 chars)
	Dynamic viewport height (mobile)	Full-screen mobile — accounts for browser chrome

SECTION 3

Fluid Typography with clamp()

```
/* clamp(minimum, preferred, maximum) */ /* Font scales fluidly between breakpoints — no media
queries needed! */ h1 { font-size: clamp(1.8rem, 5vw, 3.5rem); } h2 { font-size: clamp(1.4rem, 3vw,
2.5rem); } h3 { font-size: clamp(1.1rem, 2vw, 1.8rem); } p { font-size: clamp(1rem, 1.5vw,
1.125rem); } /* Fluid spacing */ .section { padding: clamp(40px, 8vw, 120px) clamp(16px, 5vw,
```

```
80px); } .gap { gap: clamp(16px, 3vw, 40px); } /* CSS Custom Properties (Variables) for a design system */ :root { /* Fluid type scale */ --text-sm: clamp(.875rem, 1.5vw, 1rem); --text-base: clamp(1rem, 2vw, 1.125rem); --text-lg: clamp(1.125rem,2.5vw, 1.5rem); --text-xl: clamp(1.5rem, 4vw, 2.5rem); --text-2xl: clamp(2rem, 6vw, 4rem); /* Colours (easy theming) */ --color-primary: #E44D26; --color-bg: #ffffff; --color-text: #1c1917; --color-text-muted: #78716c; /* Spacing scale */ --space-xs: 4px; --space-sm: 8px; --space-md: 16px; --space-lg: 24px; --space-xl: 48px; --space-2xl: 80px; /* Use variables everywhere */ } .card { padding: var(--space-lg); color: var(--color-text); } /* Dark mode theme — override variables only */ @media (prefers-color-scheme: dark) { :root { --color-bg: #1c1917; --color-text: #fafaf9; } }
```

SECTION 4

Responsive Images

```
<!-- Always add width and height to prevent layout shift (CLS) --> <img src="hero.jpg" alt="Mountain landscape" width="800" height="400"> <!-- srcset: browser chooses best size for screen density --> <img src="photo-800.jpg" srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1200.jpg 1200w" sizes="(max-width: 600px) 100vw, (max-width: 1024px) 80vw, 1200px" alt="Description" loading="lazy" decoding="async" width="1200" height="800" > <!-- picture: different IMAGE for different contexts (art direction) --> <picture> <!-- WebP format for modern browsers --> <source type="image/webp" srcset="hero.webp 800w, hero-2x.webp 1600w"> <!-- Landscape image for desktop --> <source media="(min-width: 1024px)" srcset="hero-landscape.jpg"> <!-- Portrait image for mobile --> <img src="hero-portrait.jpg" alt="Hero image" width="800" height="600"> </picture> <!-- CSS for always-responsive images --> img { max-width: 100%; height: auto; display: block; }
```

SECTION 5

Responsive Layout Patterns

```
/* === STACK (1-col mobile → 2-col desktop) === */ .layout { display: flex; flex-direction: column; gap: 24px; } @media (min-width: 768px) { .layout { flex-direction: row; } } /* === AUTO-GRID (no media queries!) === */ .auto-grid { display: grid; grid-template-columns: repeat(auto-fill, minmax(min(280px, 100%), 1fr)); gap: 24px; } /* === SIDEBAR (responsive: sidebar collapses on mobile) === */ .with-sidebar { display: grid; grid-template-columns: 1fr; /* mobile: 1 column */ gap: 24px; } @media (min-width: 900px) { .with-sidebar { grid-template-columns: 260px 1fr; } } /* === CONTAINER with max-width === */ .container { width: 100%; max-width: 1200px; margin-inline: auto; /* same as margin: 0 auto */ padding-inline: clamp(16px, 5vw, 80px); }
```

■ PRACTICE Q & A

Q1. What is the viewport meta tag and what happens without it?

A: *<meta name='viewport' content='width=device-width, initial-scale=1.0'> tells mobile browsers to use the actual device width. Without it, mobile browsers display the full desktop layout zoomed out — tiny and unusable.*

Q2. What is the difference between em and rem units?

A: *em: relative to the PARENT element's font-size — compounds through nesting. rem: relative to the ROOT (html) element's font-size — consistent everywhere. Use rem for font-sizes and spacing to avoid compounding surprises.*

Q3. What does clamp(1rem, 2.5vw, 2rem) do?

A: *Creates a fluid value with limits. Minimum: 1rem (never smaller). Preferred: 2.5vw (scales with viewport width). Maximum: 2rem (never larger). Font size smoothly scales between breakpoints without media queries.*

Q4. What is the srcset attribute and why is it better than just src?

A: *srcset provides multiple image files at different sizes. The browser automatically downloads the most appropriate size based on screen width and pixel density — saving bandwidth on mobile and ensuring crisp images on retina displays.*

Q5. What is a CSS custom property (variable) and what is its main benefit?

A: *CSS custom properties (--name: value) store reusable values in :root. Change the variable once and every element using var(--name) updates automatically. Essential for consistent design systems and easy dark mode implementation.*

■ Responsive Design Cheat Sheet	
`<meta name='viewport'>`	Required — enable mobile rendering
`@media (min-width: 768px)`	Tablet breakpoint (mobile-first)
`@media (min-width: 1024px)`	Desktop breakpoint
`clamp(min, pref, max)`	Fluid value with limits
`rem`	Font/spacing — relative to root
`vw / vh`	Viewport width / height units
`:root { --var: value; }`	Define CSS custom property
`var(--variable-name)`	Use CSS custom property
`auto-fill, minmax(280px,1fr)`	Auto-responsive grid columns
`max-width: 100%; height:auto;`	Responsive images in CSS
`loading='lazy'`	Lazy-load offscreen images
`@media (prefers-color-scheme:dark)`	Dark mode media query